

PSWE-03

Construcción y Mantenimiento de Software

Práctica 2

Ecosistema #2: Calidad de Código

pytest + coverage.py + Ruff + mypy + SonarQube

Documento de apoyo para presentación grupal

Fuente académica base: materiales de la Práctica 2 y Clase #2 del curso.

1. Contexto de la práctica

La Práctica 2 solicita investigar varios ecosistemas de construcción de software y determinar su **alcance, utilidad, límites y aplicación**. Dentro de esa lista, el ecosistema #2 corresponde a calidad de código y está compuesto por: **pytest**, **coverage.py**, **Ruff**, **mypy** y **SonarQube**.

Este ecosistema se relaciona directamente con la construcción profesional de software porque apoya la creación de código que no solo funcione, sino que también pueda entenderse, probarse, modificarse y mantenerse. Además, permite medir aspectos clave como pruebas creadas, porcentaje de éxito, complejidad, duplicación y cobertura de pruebas.

Resumen ejecutivo: El ecosistema de calidad de código permite verificar el comportamiento del software, medir cobertura, aplicar estándares de estilo, detectar errores de tipado y analizar mantenibilidad y seguridad. Su objetivo es reducir defectos antes de integrar o liberar cambios.

2. Descripción general del ecosistema

El ecosistema de calidad de código combina herramientas que trabajan en diferentes niveles de verificación:

- **pytest:** valida el comportamiento esperado mediante pruebas automatizadas.
- **coverage.py:** mide qué partes del código fueron ejecutadas por las pruebas.
- **Ruff:** revisa estilo, errores comunes, imports, convenciones y formato.
- **mypy:** detecta errores de tipos antes de ejecutar el programa.
- **SonarQube:** analiza calidad, mantenibilidad, duplicación, bugs, vulnerabilidades y code smells.

3. Rol de cada herramienta

Herramienta	Rol principal	Qué detecta o mide	Evidencia generada	Comando típico
pytest	Pruebas automatizadas	Errores funcionales y casos no cubiertos por lógica esperada	Resultado de pruebas: passed/failed	pytest
coverage.py	Cobertura de pruebas	Líneas y ramas no ejecutadas por las pruebas	Reporte de cobertura porcentual	coverage run -m pytest coverage report
Ruff	Linting y formato	Código mal formateado, imports no usados, errores de estilo y reglas incumplidas	Lista de advertencias o correcciones	ruff check . ruff format .
mypy	Tipado estático	Uso incorrecto de tipos, parámetros o retornos	Errores de tipos antes de ejecutar	mypy .
SonarQube	Análisis estático integral	Bugs, vulnerabilidades, code smells, duplicación y métricas de calidad	Dashboard de calidad y quality gate	sonar-scanner

4. Alcance

El alcance de este ecosistema se concentra en la verificación técnica del código durante la etapa de construcción. No se limita a ejecutar pruebas; también revisa cobertura, estilo, estándares, tipos, duplicación, mantenibilidad y posibles riesgos de seguridad.

Puede aplicarse tanto en el entorno local del desarrollador como en un flujo automatizado de integración continua. En un proyecto Python, permite revisar el código antes de hacer merge, antes de entregar una práctica o antes de publicar una versión.

Su foco principal es la calidad interna del software: claridad, legibilidad, prueba, mantenibilidad y reducción de defectos.

5. Utilidad

La principal utilidad es detectar problemas temprano, cuando corregirlos es más barato y menos riesgoso. pytest ayuda a confirmar que el sistema cumple las reglas esperadas; coverage.py permite saber qué partes del código aún no están probadas; Ruff mejora consistencia y legibilidad; mypy reduce errores relacionados con tipos; y SonarQube ofrece una visión global de mantenibilidad, seguridad y deuda técnica.

Este ecosistema también fortalece la revisión de código, porque entrega evidencia objetiva. En lugar de decir “el código se ve bien”, el equipo puede presentar pruebas ejecutadas, porcentaje de cobertura, ausencia de errores de linting, validación de tipos y análisis estático.

Para una entrega académica, ayuda a demostrar que el trabajo no solo fue programado, sino construido con criterios profesionales.

6. Límites

Estas herramientas no reemplazan el entendimiento del requerimiento ni la validación con el usuario. Un sistema puede tener 100% de cobertura y aun así implementar una regla de negocio incorrecta si la prueba fue mal diseñada.

La cobertura de pruebas mide ejecución, pero no necesariamente calidad de las pruebas. Por eso, deben incluirse casos normales, casos límite y casos inválidos.

Ruff y SonarQube pueden generar advertencias que requieren interpretación humana; no todos los hallazgos tienen la misma prioridad. mypy depende de que el código use anotaciones de tipo de forma consistente. SonarQube puede requerir instalación, servidor, configuración de proyecto y tiempo adicional de mantenimiento.

En proyectos muy pequeños, usar todo el ecosistema puede parecer pesado. Sin embargo, como ejercicio académico, es valioso porque muestra una visión profesional de construcción y mantenimiento.

7. Aplicación práctica

El flujo recomendado inicia cuando el desarrollador crea o modifica una funcionalidad. Primero ejecuta pytest para validar reglas funcionales; luego coverage.py para revisar qué parte del código está cubierta; después Ruff para limpiar estilo y formato; posteriormente mypy para validar tipos; y finalmente SonarQube para revisar calidad global.

En un equipo, este flujo puede convertirse en criterio de aceptación técnico: una tarea se considera lista cuando las pruebas pasan, la cobertura mínima se cumple, no hay errores de Ruff, mypy no reporta errores críticos y SonarQube no rompe el quality gate.

Este enfoque se puede ejecutar localmente y también integrarse en GitHub Actions, GitLab CI o Jenkins para automatizar la validación antes de integrar cambios.

8. Instalación básica y comandos de uso

Instalación sugerida para un proyecto Python:

```
python -m pip install pytest coverage ruff mypy
```

Comandos básicos para ejecutar el ecosistema:

Comando	Propósito
pytest	Ejecuta las pruebas automatizadas.
coverage run -m pytest	Ejecuta las pruebas midiendo cobertura.
coverage report	Muestra el porcentaje de cobertura en consola.
coverage html	Genera un reporte HTML de cobertura.
ruff check .	Revisa problemas de estilo y reglas de calidad.
ruff format .	Formatea automáticamente el código Python.
mypy .	Ejecuta validación de tipos estáticos.
sonar-scanner	Envía el análisis del proyecto a SonarQube cuando está configurado.

9. Ejemplo aplicado: módulo de matrícula

Si se aplica este ecosistema a un módulo de cálculo de matrícula, las herramientas se usarían así:

- **pytest:** crear pruebas para validar descuento de estudiante nuevo, estudiante regular, beca, modalidad virtual, modalidad presencial y descuento de diciembre.
- **coverage.py:** confirmar que las reglas principales, validaciones y casos límite fueron ejecutados por las pruebas.
- **Ruff:** detectar imports no usados, nombres poco claros, formato inconsistente o malas prácticas simples.
- **mypy:** verificar que monto sea numérico, mes sea entero y que las funciones retornen el tipo esperado.
- **SonarQube:** detectar duplicación, funciones demasiado grandes, complejidad innecesaria, vulnerabilidades y problemas de mantenibilidad.

10. Criterios de aceptación técnicos recomendados

- Todas las pruebas unitarias pasan correctamente.
- Cobertura mínima sugerida: 80% o superior para módulos críticos.
- No existen errores críticos de Ruff.
- El código mantiene nombres claros y formato consistente.
- mypy no reporta errores de tipos en funciones principales.
- SonarQube no reporta bloqueos críticos en seguridad, confiabilidad o mantenibilidad.
- Los resultados de pruebas y cobertura se guardan como evidencia de la construcción.

11. Guía rápida para exposición grupal

Momento	Idea para decir	Tiempo sugerido
Inicio	"Nos correspondió el ecosistema de calidad de código, compuesto por pytest, coverage.py, Ruff, mypy y SonarQube. Su objetivo es verificar que el código sea correcto, entendible, medible y mantenible."	30 s
Alcance	"Este ecosistema cubre pruebas, cobertura, estilo, tipado estático y análisis integral de calidad."	30 s
Herramientas	"pytest prueba comportamiento; coverage mide lo probado; Ruff revisa estilo; mypy valida tipos; y SonarQube ofrece una visión completa de calidad y seguridad."	1 min
Límites	"No reemplaza la validación del negocio ni la revisión humana. Además, cobertura alta no significa automáticamente pruebas buenas."	30 s
Aplicación	"En un proyecto Python se ejecutaría antes de integrar cambios y puede automatizarse en CI/CD."	30 s
Cierre	"La principal ventaja es que convierte la calidad en evidencia: pruebas, cobertura, reglas, tipos y análisis de mantenibilidad."	30 s

12. Glosario breve y fuentes oficiales

Herramienta	Descripción	Instalación / descarga	Fuente oficial
pytest	Framework de pruebas para Python.	pip install pytest	https://pytest.org/
coverage.py	Herramienta para medir cobertura de código Python.	pip install coverage	https://coverage.readthedocs.io/
Ruff	Linter y formateador rápido para Python.	pip install ruff	https://docs.astral.sh/ruff/
mypy	Validador de tipos estáticos para Python.	pip install mypy	https://mypy-lang.org/
SonarQube	Plataforma de análisis estático, calidad y seguridad.	Descarga desde SonarSource / uso con sonar-scanner	https://www.sonarsource.com/products/sonarqube/

13. Conclusión

El ecosistema de calidad de código es una base esencial para construir software con disciplina técnica. Su aporte principal es transformar la calidad en algo verificable mediante pruebas, cobertura, reglas de estilo, tipos y análisis estático. Aunque no sustituye la validación del negocio ni la revisión humana, sí permite reducir defectos, mejorar la mantenibilidad y entregar evidencia objetiva del estado del código.

14. Referencias

1. pytest Documentation. <https://pytest.org/>
2. Coverage.py Documentation. <https://coverage.readthedocs.io/>
3. Ruff Documentation. <https://docs.astral.sh/ruff/>
4. mypy Documentation. <https://mypy-lang.org/>
5. SonarQube Product Documentation / SonarSource. <https://www.sonarsource.com/products/sonarqube/>
6. Material del curso PSWE-03: Práctica 2 y Clase #2..